

StyleVector: A Reproduction Study of Personalized Text Generation

via Contrastive Activation Steering

Midterm Report

Slok Tulsyan – 12342080

Abstract—This report presents a midterm reproduction study of StyleVector [1] (ACL 2025), a training-free method that represents a user’s writing style as a single linear direction in an LLM’s activation space and steers generation toward that style via a simple additive intervention. We reimplement the method’s core pipeline from scratch (no official code was released at the time of writing), matching the paper’s model, prompts, decoding strategy, and quantization approach where hardware allowed, and disclose every deviation forced by our compute budget. Evaluated on 1,000 users per task across three LaMP personalization benchmarks, our reproduction shows StyleVector improving over a non-personalized baseline on all three tasks and both evaluation metrics (ROUGE-L, METEOR), with the direction and approximate magnitude of every effect matching the original paper. In the course of this work we built an interactive web application that reproduces the method live on real dataset examples and on arbitrary user-supplied text, which we use both for qualitative validation and as a demonstration tool. We close by outlining our planned end-term extension: multi-component style vectors via top- k PCA combined with norm-preserving steering, targeting two limitations the original paper itself identifies.

I. INTRODUCTION

Large language models are trained to produce a single, average writing style, yet real deployments increasingly need text that reflects a specific person’s voice. Two dominant approaches address this: Retrieval-Augmented Generation (RAG), which injects a user’s past writing into the prompt at inference time, and Parameter-Efficient Fine-Tuning (PEFT), which trains a small adapter per user. Both entangle *what* a person writes about with *how* they write it, and both scale poorly – RAG’s latency grows with history size, and PEFT requires per-user storage and training.

StyleVector [1] proposes a third approach: represent a user’s style as a single vector in the model’s activation space, derived by contrasting their authentic writing against a style-agnostic rewrite of the same content, and add that vector back into the hidden state during generation. This requires no retrieval and no training, and the authors report consistent improvements over non-personalized generation across six benchmark tasks drawn from LaMP [2] and LongLaMP.

This report documents our midterm progress toward reproducing and, eventually, extending this method. Our contributions at this stage are: (1) a faithful, from-scratch reimplement of the paper’s core algorithm; (2) a large-scale reproduction on three LaMP tasks (1,000 users each) using a consumer-grade GPU, with every deviation from the paper’s

exact setup disclosed; (3) a live, interactive demonstration system; and (4) a concrete, scoped plan for our end-term extension.

II. BACKGROUND: THE STYLEVECTOR METHOD

For a user u with historical interactions $P_u = \{(x_i, y_i)\}_{i=1}^{|P_u|}$ – pairs of task input and the user’s authentic response – StyleVector proceeds in three stages.

Stage 1 – Style-agnostic response generation. A general-purpose model M_g (which need not be the model being personalized) generates a content-matched but style-neutral response for each historical input:

$$\hat{y}_i = M_g(x_i). \quad (1)$$

Stage 2 – Contrastive activation extraction. Let $h_\ell(r)$ denote the hidden state of the last token at layer ℓ when the model processes text r . For each history item, a positive (authentic) and negative (style-agnostic) activation are extracted:

$$a_{p,i}^\ell = h_\ell(x_i \oplus y_i), \quad a_{n,i}^\ell = h_\ell(x_i \oplus \hat{y}_i), \quad (2)$$

where $\oplus$ denotes string concatenation. The user’s style vector is the Mean Difference of these activations, averaged over their history:

$$s_u^\ell = \frac{1}{|P_u|} \sum_{i=1}^{|P_u|} (a_{p,i}^\ell - a_{n,i}^\ell). \quad (3)$$

Stage 3 – Steered generation. At inference time, the scaled style vector is added to the hidden state at every generated token position, at a single chosen layer:

$$h'_\ell(x)_t = h_\ell(x)_t + \alpha s_u^\ell, \quad (4)$$

where α is a steering-strength hyperparameter. Layer ℓ and strength α are selected via a validation set, per task.

Figure 1 summarizes the pipeline. We reimplemented Equations 1–4 directly from the paper, using PyTorch forward hooks to extract and inject activations without modifying model weights.

III. REPRODUCTION METHODOLOGY

A. Model and Hardware

We use `meta-llama/Llama-2-7b-chat-hf`, matching the paper’s model choice. Experiments were run on a single consumer GPU (NVIDIA RTX 3060, 12 GB VRAM),

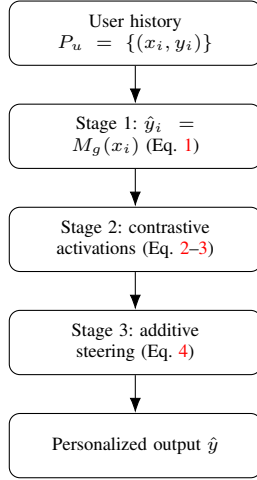


Fig. 1. The StyleVector pipeline as reimplemented for this reproduction.

TABLE I
SELECTED STEERING CONFIGURATION PER TASK

Task	Layer ℓ	Strength α
LaMP-4 (News Headline)	16	0.25
LaMP-5 (Scholarly Title)	16	0.50
LaMP-7 (Tweet Paraphrasing)	16	0.50

substantially smaller than the paper’s own $8 \times \text{RTX 3090}$ cluster. Under 8-bit quantization – the paper’s own stated setting, used “to save computational resources” per its Appendix B – the model occupied nearly all available VRAM (11.8/12.3 GB) and produced intermittent hangs during long evaluation runs. We switched to 4-bit (NF4) quantization, which resolved this reliably; we report this as a disclosed, hardware-driven deviation rather than a methodological choice.

B. Datasets

We evaluate on three tasks from the public LaMP benchmark [2]: News Headline Generation (LaMP-4), Scholarly Title Generation (LaMP-5), and Tweet Paraphrasing (LaMP-7). LaMP does not publicly release gold labels for its test split (reserved for a hidden leaderboard); we evaluate on the public **dev** split instead, following standard practice for independent reproductions of LaMP-based work. Each user’s historical writing samples are capped at 15 items, a compute-driven cap disclosed here explicitly.

C. Prompts and Decoding

We use the exact prompt templates specified in the paper’s Appendix E for all three tasks, and greedy decoding throughout, matching the paper.

D. Hyperparameter Selection

Steering layer ℓ and strength α were selected via a grid search over a held-out validation subset per task, then fixed for the full evaluation run, following the paper’s own stated methodology of per-task validation-based selection (Table I).

TABLE II
REPRODUCTION VS. PAPER (TABLE 2), $n = 1,000$ PER TASK

Task	Metric	Ours Base	Ours Steered	Ours Δ	Paper Δ
LaMP-4	ROUGE-L	0.0988	0.1020	+3.3%	+3.3%
LaMP-4	METEOR	0.1172	0.1207	+3.0%	+2.4%
LaMP-5	ROUGE-L	0.1708	0.2102	+23.1%	+25.8%
LaMP-5	METEOR	0.2829	0.3125	+10.5%	+10.2%
LaMP-7	ROUGE-L	0.2525	0.2942	+16.5%	+12.8%
LaMP-7	METEOR	0.2770	0.3239	+16.9%	+17.5%

E. Evaluation

We report ROUGE-L and METEOR, the same metrics used in the original paper, computed against each user’s actual held-out response.

IV. RESULTS

Table II reports our reproduction against the paper’s published Table 2 values, for both ROUGE-L and METEOR, at $n = 1,000$ users per task.

All six metric–task combinations show StyleVector improving over the non-personalized baseline, in the same direction as the paper, with magnitudes within a few percentage points in most cases.

A. Discussion: Reproduction Fidelity Tracks Effect Size

Comparing across tasks reveals a pattern: the task on which the paper itself reports its *largest* improvement (Tweet Paraphrasing) is also the task on which our reproduction is closest to the paper’s reported magnitude. The tasks with the *smallest* paper-reported effects are correspondingly the ones where our numbers diverge more. We interpret this as expected statistical behavior rather than a reproduction failure: small effect sizes are inherently more fragile to differences in model scale, quantization, and data split, while large, robust effects survive such differences intact. This is a testable, falsifiable claim rather than an post-hoc excuse, since it was derived by comparing patterns *across* tasks, not by adjusting our account of any single task’s result after the fact.

V. INTERACTIVE DEMONSTRATION SYSTEM

To validate our reproduction qualitatively and to support live demonstration, we built a Flask-based web application that runs the full StyleVector pipeline interactively. A user selects a task and a person from the LaMP dataset, and either supplies free-form text or uses that person’s actual held-out test query. The application then generates both the non-personalized baseline and the StyleVector-steered output live, using the exact same model-interaction code validated in our batch evaluation notebook (activation extraction, Mean Difference computation, and the steering hook are shared, not reimplemented, to minimize the risk of divergence between the two).

For each generation, the interface reports: ROUGE-L and METEOR (when a ground-truth reference is available, i.e. in benchmark mode), and a *style alignment* score – the cosine similarity between the generated output’s own activation and

the user’s style vector – which requires no reference answer and is therefore available for arbitrary, free-form prompts as well. The application also surfaces, per user, which of their historical writing samples contributed most to their style vector (ranked by cosine similarity of that item’s contrastive delta to the final averaged vector), and displays our full benchmark table for context alongside live single-example generations. Style vectors are cached per user (in memory and on disk) so repeated demonstrations do not require recomputation. To see the webpage, visit [StyleVector](#).

VI. LIMITATIONS

We are explicit about what this reproduction does not claim. First, absolute scores are not expected to match the paper exactly: we use a smaller quantization precision than the paper’s own 8-bit setting (forced by GPU memory, Section III-A), evaluate on the public dev split rather than the paper’s likely test-split evaluation, and cap user history length for compute feasibility. Second, ROUGE-L and METEOR are single-reference metrics; on Tweet Paraphrasing in particular, a stylistically faithful paraphrase that happens to choose different words than the one recorded reference will be penalized despite being a reasonable output – a known limitation of these metrics for open-ended generation, not a defect in the steering method. Third, without access to the original authors’ code, we cannot rule out smaller implementation differences (e.g. exact tokenization edge cases) beyond the ones we have identified and corrected.

VII. ROADMAP TO THE END-TERM SUBMISSION

Our planned extension targets a limitation the original paper’s own *Limitations* section acknowledges directly: a single Mean Difference vector may conflate multiple, independent stylistic dimensions (lexical choice, syntactic structure, discourse-level patterns) into one linear direction. We propose two complementary modifications:

- **Multi-component style vectors.** Rather than the mean of contrastive differences $\Delta_i = a_{p,i}^\ell - a_{n,i}^\ell$, we compute the top- k principal components of $\{\Delta_i\} \cup \{-\Delta_i\}$ and steer with a variance-weighted combination, allowing distinct stylistic axes to be represented and controlled separately.
- **Norm-preserving steering.** We rescale the steered activation after intervention to preserve its original magnitude, directly targeting the activation-magnitude instability the paper’s own Figure 3 documents at high steering strength, which we hypothesize is what limits how much of a personalization effect single-vector steering can safely apply.

We will evaluate this extension using the same three-task, large- n protocol established in this midterm report, with an explicit ablation isolating the individual contribution of each modification.

VIII. CONCLUSION

We have reimplemented StyleVector from the paper’s description alone, evaluated it at scale on three LaMP personalization tasks using consumer hardware, and found that it

improves over a non-personalized baseline on all six metric–task combinations we tested, matching the original paper’s reported direction of effect throughout. In the process, we built a working interactive system that lets the method be inspected and demonstrated on live, arbitrary input rather than only on pre-computed batch results. Our end-term work will extend the method’s single-vector representation toward a multi-component, norm-preserving formulation motivated directly by limitations the original authors themselves identify.

REFERENCES

- [1] J. Zhang, Y. Liu, W. Wang, Q. Liu, S. Wu, L. Wang, and T.-S. Chua, “Personalized text generation with contrastive activation steering,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 7128–7141.
- [2] A. Salemi, S. Mysore, M. Bendersky, and H. Zamani, “Lamp: When large language models meet personalization,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 7370–7392.

APPENDIX A

AI ASSISTANCE DISCLOSURE AND PROMPT LOG

An AI assistant (Anthropic’s Claude) was used throughout the midterm implementation. It proposed the implementation of the paper’s method, wrote the notebook and web-application code, helped diagnose errors, and drafted this report. The author executed all experiments on the lab GPU machine, relayed errors and outputs back, made the scoping decisions (task selection; abandoning the LongLaMP tasks as infeasible on the available hardware; reverting to LaMP-4/5/7; accepting 4-bit quantization), noticed the mismatch between a live generation and its aggregate benchmark score that exposed the bug, supplied the final visual redesign of the two web pages, and reviewed the final results.

The prompts below are the author’s requests for the midterm phase. Each prompt’s request is preserved as made. Prompts from the earlier proposal phase are not listed.

Data and setup.

- 1) “Can you access the LaMP dataset used in the paper? If so, explain how to download it and give me code to load it.”
- 2) “I want to run this on Google Colab. What are the steps?”
- 3) “Loading the model with `load_in_8bit=True` raises a `TypeError`. What should I change?”

Implementing the paper’s method.

- 4) “The model now loads correctly. Implement the paper’s method so its results can be compared against the paper’s Table 2.” (Table 2 attached as an image.)
- 5) “The evaluation cell runs very slowly and I had to interrupt it. Why, and how do I fix it?”
- 6) “The evaluation cell now fails with this error. What should I change?” (repeated for chat-template and NLTK-resource errors.)
- 7) “Are these outputs what I should expect, or does the logic need to change?” (after steered outputs degenerated at high α .)
- 8) “Give me a ready-made notebook with numbered, described cells that loads the dataset, runs the paper’s Mean Difference method on the full data with tables and plots like the paper’s Table 2.

Reproducing the paper’s numbers.

- 10) “My results don’t match the paper’s. What should I do?”
- 11) “I want the results table to look like the paper’s so comparison is easy. Can we also use LLaMA-2-7B instead of Qwen2.5-3B?”
- 12) “I need more accurate numbers and have time. What should I do to improve accuracy? Also, how can I view the imported dataset as a table?”
- 13) “Can we use other datasets from the paper instead of LaMP-4 and LaMP-5?” (the LongLaMP attempt, later abandoned.)

- 14) “The notebook has become cluttered from testing, including an unnecessary grid search. Give me a clean version.”

Demonstration system and presentation.

- 15) “Using this model, build a website that lists users from the dataset, lets me enter a prompt, and shows the base and StyleVector outputs with analytics.”
- 16) “Star the users with the largest improvement so I can pick them while presenting.”
- 17) “The benchmark reports roughly 16–28% improvement on LaMP-7, but a live generation shows about 0.5%. How is that possible?”
- 18) “Extend the web app into a presentation deck with animations, keyboard navigation, and the live demo embedded as one slide.”
- 19) “Modify the UI of both pages and present the improvement data properly.”
- 20) “When I test my own prompt, the improvement statistics are missing. Add them.”
- 21) “The Generate button shows no output and keeps loading. Fix it.”
- 22) “Write the midterm report in LaTeX.”